RiskLab technical design

Monte Carlo portfolio risk simulation engine

Project owner Aiden Federico-Dyer | Version 0.1 | September 24 2026

Status: design proposal; implementation not started. The model and tests below are proposed; no completed simulations or benchmarks are claimed.

Purpose and first version

I plan to build a local tool that shows how a portfolio could behave under a declared return model. The user will provide a holdings snapshot, a cash balance, initial prices, and a return history. The output will show a distribution of portfolio outcomes, estimated loss measures, and path drawdowns together with the assumptions that produced them.

The first version will use correlated daily Gaussian log returns and fixed share quantities. It will model one to ten assets in USD over up to 252 sessions, with a configurable path count and seed. A zero-mean log-return scenario will be available beside estimated historical log means to make drift sensitivity visible.

Requirements

ID	Required behavior	Completion evidence
R1	Validate aligned returns and covariance	Invalid covariance rejected before sampling
R2	Preserve input asset order in holdings and shocks	Permutation fixture leaves portfolio values unchanged
R3	Make runs repeatable in the pinned environment	Same inputs and seed reproduce arrays
R4	Separate terminal loss from path drawdown	Different measures agree with hand fixtures
R5	Report uncertainty and assumptions	Sample count, seed, horizon, model and errors exported

Model boundary

Compute log returns over complete adjacent sessions shared by every asset. Reject missing prices and multi-session gaps; do not forward-fill them.

For initial synthetic fixtures there are no dividends or splits. A later total-return input mode may model each asset as a reinvested total-return index scaled to its initial position value; it must not label those index paths as raw tradable share prices. Actual cash dividend and split processing is outside the first model.

Design choices

NumPy sampling with explicit metadata. NumPy supports multivariate Gaussian sampling and requires a symmetric positive-semidefinite covariance matrix [1]. The implementation will validate that requirement and record the generator, factorization method, versions, and seed. Repeatability will be promised only for the pinned execution environment.

Fixed holdings first. Fixed shares make valuation easier to verify. Rebalancing and trading require a separate execution policy.

Architecture and data contracts

The estimator will be separate from the sampler so manually specified scenarios can be tested without historical calibration. Portfolio valuation will consume simulated prices and fixed holdings; it will not refit the model inside each path.

Module responsibilities

Module	Responsibility	Main output
inputs.py	Validate holdings, units, dates and price basis	PortfolioSnapshot and ReturnMatrix
calibration.py	Estimate log means and sample covariance	ModelParameters
simulation.py	Generate correlated log increments in batches	ScenarioBatch
valuation.py	Mark fixed holdings and cash along each path	Equity paths and running peaks
risk.py	Aggregate losses, drawdowns and sampling errors	RiskSummary

Proposed records

Record	Required fields and conventions
PortfolioSnapshot	as_of_session, currency, cash, ordered symbols, quantity, initial_price
ModelParameters	daily_log_mean vector, daily_log_cov matrix, estimation_dates, model_id
SimulationConfig	n_paths, n_steps, seed, generator_id, batch_size, drift_mode
RiskSummary	initial_equity, loss_quantiles, expected_shortfall, probability_of_loss, drawdown_quantiles
RunManifest	input and config hashes, return_basis, asset_order, versions, status

Proposed interface

```
fit_model(returns, asset_order, drift_mode) -> ModelParameters
simulate(snapshot, model, config) -> RiskResult
summarize(terminal_equity, max_drawdowns, initial_equity) -> RiskSummary
```

Every matrix row will represent one session and every column one asset. Symbol matching must occur by identifier before converting to arrays. The CLI will reject duplicate symbols, insufficient observations, nonfinite values, negative cash or quantities, nonpositive prices, and nonpositive initial equity.

The output folder will contain manifest.json, model_parameters.json, summary.json, and terminal_outcomes.csv. A reproducibly selected small subset of paths may be exported for charting. This preview subset must never be used to calculate the summary statistics.

Simulation and risk definitions

Correlated return model

For asset i and session step t, the proposed model is:

$$g_t \sim N(m, \Sigma), \qquad S_{i,t+1} = S_{i,t}\exp(g_{i,t})$$

m and covariance Sigma are daily log-return parameters. There is no additional subtraction of one-half variance here because m already represents mean log return. A separate future mode accepting arithmetic GBM drift would require that conversion explicitly. Daily parameters will not be annualized and then accidentally used as daily inputs.

The generator will draw independent vectors over time with within-vector correlation across assets. That is a modeling assumption: the first version does not reproduce volatility clustering, jumps, or serial dependence. Cash earns zero interest; each path is valued as cash plus the sum of quantity times price.

Covariance policy

Estimate sample covariance with denominator n - 1 after aligning complete observations. Check finiteness, symmetry, and eigenvalues. Reject a materially negative eigenvalue. Tiny negative values within a declared relative numerical tolerance may be clipped to zero, with the before-and-after minimum eigenvalues recorded. Singular positive-semidefinite matrices are allowed through an eigenvalue factorization; ordinary Cholesky is not required [1].

Loss measures

Terminal dollar loss is initial equity minus terminal equity, so gains appear as negative losses. At confidence alpha, empirical VaR is the loss at sorted index ceil(alpha times N). Expected shortfall is the mean of the largest ceil((1 - alpha) times N) losses. The chosen finite-sample conventions will be exported; losses will not be floored at zero.

Probability of loss is the proportion of paths with terminal equity strictly below initial equity. Its approximate Monte Carlo standard error is the square root of p times (1 - p) divided by N. Maximum drawdown is calculated within each full path from its own running peak and then summarized across paths. It is not the drawdown of the median path.

Worked acceptance fixtures

For initial equity $1,000 and terminal values $800, $900, $1,000, $1,100, $1,200, sorted losses are -$200, -$100, $0, $100, $200. At 80% confidence, VaR is $100 and expected shortfall is $200 under the declared convention. Probability of loss is 2/5, or 40%. This deliberately tiny example validates indexing; it is unsuitable for estimating tail risk.

A separate path 1000, 1200, 900, 1100 has maximum drawdown 25%, despite ending 10% above its initial value.

Validation and operational limits

Acceptance tests

Test	Fixture	Expected behavior
Zero uncertainty	Zero covariance and zero log mean	Every price and equity remains constant
Deterministic growth	Zero covariance, nonzero log mean	Values equal initial price times exp(steps times mean)
Dependence	Two assets with identical shocks and returns	Matching percentage changes along every path
Asset alignment	Permute both model and holdings columns	Portfolio outcomes unchanged within tolerance
Covariance error	Clearly indefinite covariance matrix	Reject with model-validation error
Repeatability	Same inputs, seed, versions and batch size	Same generated arrays and summary
Loss convention	Five terminal values on page 3	VaR $100, ES $200, loss probability 40%
Path risk	1000, 1200, 900, 1100	Maximum drawdown 25%

Tests are planned, not executed. Statistical checks will additionally compare large-sample generated means and covariance with requested parameters using tolerances tied to sampling error. Such tests must not fail simply because a random draw differs slightly from its expectation.

Sampling precision

For a Bernoulli probability estimate, 10,000 independent paths have a worst-case standard error of 0.5 percentage points. That statement is about Monte Carlo sampling, not whether the return model is correct. Tail estimates need separate uncertainty analysis; the first release will require at least 100 observations in the selected tail and flag materially unstable estimates across independent seeds.

Memory and runtime budget

A full 10,000 by 253 by 10 float64 price tensor alone contains about 202.4 MB of values, excluding overhead. The proposed implementation will instead process batches, track current prices, running peaks, and maximum drawdowns, and retain terminal values plus selected display paths. The seed and batch size will both be recorded because random-number consumption can depend on batching.

The initial engineering target is 10,000 paths, ten assets and 252 steps within 15 seconds and 300 MB on a documented laptop. It is unmeasured. The first benchmark will identify hardware, library versions, thread settings, wall time, and peak memory.

Failure behavior

Reject invalid inputs before generation. Stop on numeric overflow or nonfinite simulated values and identify the scenario batch. A cancelled or failed run will never publish a complete summary. Users may inspect a failure report, but partial paths will be labeled incomplete.

Implementation sequence and review

Milestones with completion evidence

Step	Work to implement	Evidence before moving on
1	Fixed-path valuation and risk measures	Terminal-loss and drawdown fixtures reconcile
2	One-asset random increments	Seeded reproduction and zero-volatility behavior
3	Correlated multi-asset generator	Covariance and asset-order tests
4	Historical estimation and input validation	Complete-session alignment and model diagnostics
5	Batched execution and visual report	Same-path reference comparison and measured memory

The first demonstration will compare deterministic, low-volatility, and high-volatility scenarios for identical holdings. Show the assumptions and seed; plotted extremes are simulated observations, not guaranteed bounds.

Integration boundary

PortfolioLab may export a target allocation that is converted to an explicit hypothetical snapshot. BacktestLab may export actual simulated holdings at a chosen date. RiskLab will identify the snapshot and its units: shares, index exposure, or cash.

Options positions will initially be rejected. Revaluing options under scenarios requires OptionsLab and consistent dates, volatility assumptions, and contract multipliers. It cannot be approximated by treating an option as an ordinary stock with the same return series.

Model limitations and extensions

Gaussian log returns are an understandable baseline but can underrepresent extreme losses and changing dependence. A later block-bootstrap model could retain some temporal structure by resampling contiguous historical windows. Named stress scenarios could test specific shocks without claiming to assign them probabilities. Model comparisons will keep portfolio, horizon, and reporting conventions fixed.

Questions for a design review

1. How many complete estimation observations should be required for each asset count?
2. Should estimated drift or a declared neutral scenario be the default display?

Technical references

[1] NumPy documentation, Generator.multivariate_normal. Sampling contract, covariance requirements, factorization choices, and environment-dependent reproducibility. Read the documentation.

[2] NumPy documentation, Random Generator. Generator configuration and random-number interfaces. Read the documentation.